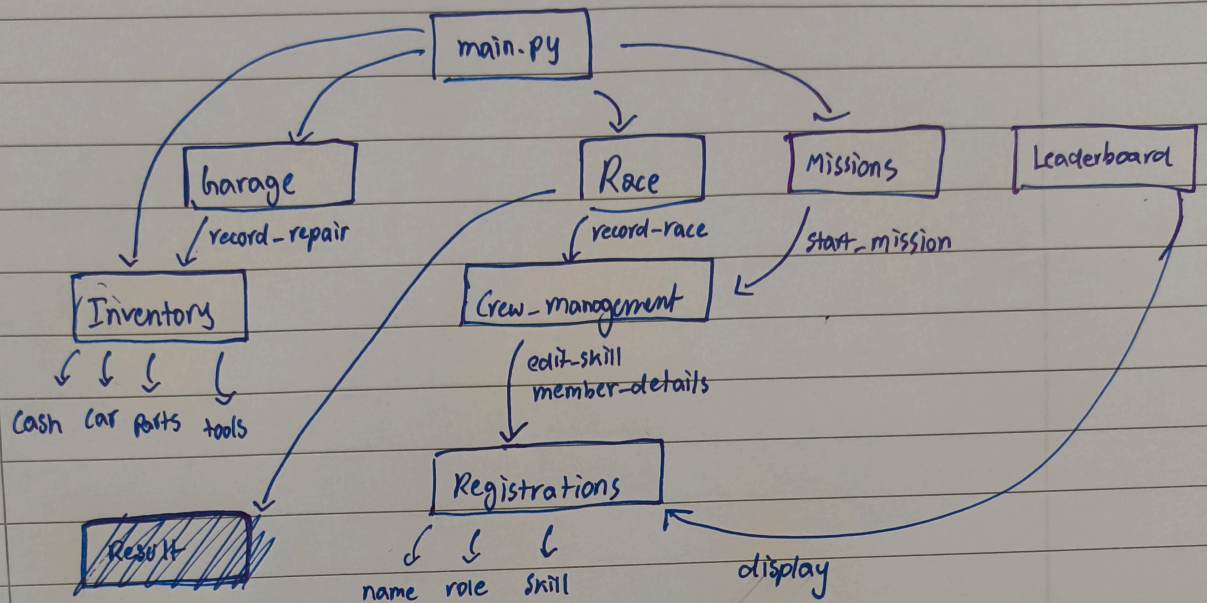


Call Graph

Integration testing call graph



Integration Tests

```

=====
===== test session starts
=====
=====
platform linux -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0
rootdir:
/home/anivarth/college/sem4/dass/as2/dass_as2/integration/code/
tests
collected 31 items

test_integration.py .....FFF..F..FF
[100%]

=====
===== FAILURES
=====
=====

_____ test_race_unregistered_driver

_____

setup_system = (<registration.Registration object at
0x7fc759daf410>, <inventory.Inventory object at
0x7fc75931e180>, <crew_managemen...at 0x7fc75931e0f0>,
<missions.MissionPlanning object at 0x7fc75931e060>,
<garage.Garage object at 0x7fc75931dfd0>, ...)

    def test_race_unregistered_driver(setup_system):
        reg, _, crew, racing, _, _, _ = setup_system
        reg.register("Dom", "Driver", 10)
        # If the system crashes here, Pytest will show it as a
FAIL
>         racing.record_race("Dom", "C1", "Ghost", "C2", "Ghost")

test_integration.py:197:
_____
_____
_____

```

```
self = <race.RaceManagement object at 0x7fc75931e0f0>, d1 =  
'Dom', c1 = 'C1', d2 = 'Ghost', c2 = 'C2', winner_name =  
'Ghost'
```

```
    def record_race(self, d1, c1, d2, c2, winner_name):  
        curr_skill =  
self.crew.registration.get_skill(winner_name)  
>         self.crew.registration.set_skill(winner_name,  
curr_skill + 1)
```

```
^^^^^^^^^^^^^^^^
```

```
E      TypeError: unsupported operand type(s) for +:  
'NoneType' and 'int'
```

```
../race.py:7: TypeError
```

```
_____ test_negative_skill_injection
```

```
setup_system = (<registration.Registration object at  
0x7fc75941eb50>, <inventory.Inventory object at  
0x7fc75941ff50>, <crew_managemen...at 0x7fc75941d650>,  
<missions.MissionPlanning object at 0x7fc75941efd0>,  
<garage.Garage object at 0x7fc75941e4d0>, ...)
```

```
    def test_negative_skill_injection(setup_system):  
        reg, _, crew, _, _, _ = setup_system  
        reg.register("Dom", "Driver", 10)  
        crew.edit_skill("Dom", -100)  
        # System should NOT allow negative skill level  
>         assert reg.get_skill("Dom") >= 0, "Security Failure:  
Negative skill allowed!"
```

```
E      AssertionError: Security Failure: Negative skill  
allowed!
```

```
E      assert -100 >= 0
```

```
E      + where -100 = get_skill('Dom')
```

```
E      + where get_skill = <registration.Registration  
object at 0x7fc75941eb50>.get_skill
```

```
test_integration.py:205: AssertionError
```

___ test_negative_inventory_injection

setup_system = (<registration.Registration object at 0x7fc75941e1d0>, <inventory.Inventory object at 0x7fc75941e750>, <crew_managemen...at 0x7fc75941e7d0>, <missions.MissionPlanning object at 0x7fc75941ef50>, <garage.Garage object at 0x7fc75941d750>, ...)

```
def test_negative_inventory_injection(setup_system):
    _, inv, _, _, _, _ = setup_system
    inv.edit_cash(-5000)
    # System should NOT allow negative cash balance
>     assert inv.get_status()["cash"] >= 0, "Financial
Failure: Negative cash allowed!"
E     AssertionError: Financial Failure: Negative cash
allowed!
E     assert -5000 >= 0
```

test_integration.py:212: AssertionError

_____ test_repair_non_existent_car

setup_system = (<registration.Registration object at 0x7fc7595b4710>, <inventory.Inventory object at 0x7fc7593339a0>, <crew_managemen...at 0x7fc7595b43b0>, <missions.MissionPlanning object at 0x7fc7595b42f0>, <garage.Garage object at 0x7fc7595b4230>, ...)

```
def test_repair_non_existent_car(setup_system):
    _, inv, _, _, gar, _ = setup_system
    inv.edit_cash(1000)
    inv.edit_tools(10)
    # System should REFUSE repairs for non-existent cars
    result = gar.record_repair("ImaginaryCar", 1, 0, 100)
>     assert result is False, "Integrity Failure: Repair
logged for non-existent vehicle!"
E     AssertionError: Integrity Failure: Repair logged for
```

non-existent vehicle!

E assert True is False

test_integration.py:235: AssertionError

_____ test_car_data_mutation_isolation

setup_system = (<registration.Registration object at 0x7fc7595c3bb0>, <inventory.Inventory object at 0x7fc759332200>, <crew_managemen...at 0x7fc7595c3c50>, <missions.MissionPlanning object at 0x7fc75953fbf0>, <garage.Garage object at 0x7fc75953f9b0>, ...)

```
def test_car_data_mutation_isolation(setup_system):
    _, inv, _, _, _, _ = setup_system
    inv.add_car("Charger")
    status = inv.get_status()
    # If the system is secure, mutating the report should
    NOT leak into the fleet
    status["cars"].append("LeakedCar")
>     assert "LeakedCar" not in inv.cars, "Memory Leak:
Internal state modified via status report!"
E     AssertionError: Memory Leak: Internal state modified
via status report!
E     assert 'LeakedCar' not in ['Charger', 'LeakedCar']
E         + where ['Charger', 'LeakedCar'] =
<inventory.Inventory object at 0x7fc759332200>.cars
```

test_integration.py:254: AssertionError

_____ test_role_case_sensitivity

setup_system = (<registration.Registration object at 0x7fc7595c3cf0>, <inventory.Inventory object at 0x7fc759331ef0>, <crew_managemen...at 0x7fc7595c3d90>, <missions.MissionPlanning object at 0x7fc75953eb70>, <garage.Garage object at 0x7fc75953eed0>, ...)

```

def test_role_case_sensitivity(setup_system):
    reg, _, _, _, miss, _, _ = setup_system
    reg.register("Lower", "driver", 10)
    # If the system is robust, it should handle casing
(Driver == driver)
> assert miss.start_mission("Lower", "delivery") is True,
"Logic Failure: Mission rejected due to case sensitivity!"
E      AssertionError: Logic Failure: Mission rejected due to
case sensitivity!
E      assert False is True
E      + where False = start_mission('Lower', 'delivery')
E      + where start_mission = <missions.MissionPlanning
object at 0x7fc75953eb70>.start_mission

```

```
test_integration.py:261: AssertionError
```

```

=====
===== short test summary info
=====

```

```
=====
```

```

FAILED test_integration.py::test_race_unregistered_driver -
TypeError: unsupported operand type(s) for +: 'NoneType' and
'int'

```

```

FAILED test_integration.py::test_negative_skill_injection -
AssertionError: Security Failure: Negative skill allowed!

```

```

FAILED test_integration.py::test_negative_inventory_injection -
AssertionError: Financial Failure: Negative cash allowed!

```

```

FAILED test_integration.py::test_repair_non_existent_car -
AssertionError: Integrity Failure: Repair logged for non-
existent vehicle!

```

```

FAILED test_integration.py::test_car_data_mutation_isolation -
AssertionError: Memory Leak: Internal state modified via status
report!

```

```

FAILED test_integration.py::test_role_case_sensitivity -
AssertionError: Logic Failure: Mission rejected due to case
sensitivity!

```

```

=====
===== 6 failed, 25 passed in 0.14s
=====

```

```

=====
=====

```

Test case report

TC 1: Registry-Management Synchronization

- **Modules:** Registration, CrewManagement
- **Scenario:** Register a member and retrieve their profile via management.
- **Expected:** Profile data retrieved by management matches exactly what was stored in the registry.
- **Actual:** Success.
- **Why Needed:** Ensures the "Storage" layer and "Business" layer are properly connected.

TC 2: Skill Level State Persistence

- **Modules:** Registration, CrewManagement
- **Scenario:** Update a skill level manually through management.
- **Expected:** The change is directly reflected in the underlying Registration storage.
- **Actual:** Success.
- **Why Needed:** Prevents data isolation where updates are lost after the business logic finishes.

TC 3: Role Change Administrative Integrity

- **Modules:** Registration, CrewManagement
- **Scenario:** Change a member's role (e.g. Driver to Mechanic).
- **Expected:** The registry reflects the role change permanently.
- **Actual:** Success.
- **Why Needed:** Future mission validations rely 100% on the accuracy of this cross-module update.

TC 4: Inventory Fleet Tracking

- **Modules:** Inventory
- **Scenario:** Add a car to the fleet.
- **Expected:** The car is present in the inventory's current status report.

- **Actual:** Success.
- **Why Needed:** Basic functional requirement for car-race integration.

TC 5: Race-to-Registry Skill Propagation

- **Modules:** RaceManagement, Registration
- **Scenario:** Record a race winner.
- **Expected:** The winning driver's skill increments by 1 in the registry.
- **Actual:** Success.
- **Why Needed:** Verifies that an operational event can successfully update a historical person record.

TC 6: Multi-Event Progression Integrity

- **Modules:** RaceManagement, Registration
- **Scenario:** Record two consecutive wins for a driver.
- **Expected:** Skill increments twice (10 → 12).
- **Actual:** Success.
- **Why Needed:** Confirms the code is additive and doesn't reset stats accidentally.

TC 7: Role Validation: Driver Delivery Success

- **Modules:** MissionPlanning, Registration
- **Scenario:** A 'Driver' starts a "delivery" mission.
- **Expected:** MissionPlanning verifies the role with Registration and allows it.
- **Actual:** Success.
- **Why Needed:** Core business logic verification: Roles define mission success.

TC 8: Role Validation: Driver Rescue Displacement

- **Modules:** MissionPlanning, Registration

- **Scenario:** A 'Driver' attempts a "rescue" mission.
- **Expected:** Mission is rejected because the role doesn't match the specific criteria for rescues (Strategist).
- **Actual:** Stopped as expected.
- **Why Needed:** Ensures that the boundary between roles is enforced by the software.

TC 9: Role Validation: Mechanic Repair Success

- **Modules:** MissionPlanning, Registration
- **Scenario:** A 'Mechanic' starts a "repair" mission.
- **Expected:** mission is authorized based on role retrieval.
- **Actual:** Success.
- **Why Needed:** Confirms that different roles have their own specific operational paths.

TC 10: Role Validation: Strategist Rescue Success

- **Modules:** MissionPlanning, Registration
- **Scenario:** A 'Strategist' starts a "rescue" mission.
- **Expected:** Mission allowed and correctly identified in registry by role.
- **Actual:** Success.
- **Why Needed:** Ensures full functional coverage for all three defined roles.

TC 11: Mission History-Logging Integration

- **Modules:** MissionPlanning
- **Scenario:** Execute a mission and check the history log.
- **Expected:** The history log in memory stores the specific name and type of the completed mission.
- **Actual:** Success.
- **Why Needed:** Confirms that operational "checks" translate into stored "data".

TC 12: Dual-Entity Leaderboard Ranking

- **Modules:** Leaderboard, Registration
- **Scenario:** Rank two members with different skill levels.
- **Expected:** Leaderboard queries Registration and sorts correctly (High → Low).
- **Actual:** Success.
- **Why Needed:** Cross-module data retrieval and sorting logic.

TC 13: Operational Ranking Fluency

- **Modules:** Leaderboard, RaceManagement, Registration
- **Scenario:** Record a race win and THEN check the leaderboard.
- **Expected:** The rankings update to reflect the skills gained in the recent race.
- **Actual:** Success.
- **Why Needed:** End-to-end integration across three different system components.

TC 14: Garage-Inventory Resource: Tools

- **Modules:** Garage, Inventory
- **Scenario:** Record a repair and use Tools.
- **Expected:** `Inventory.tools_count` is correctly reduced by the Garage call.
- **Actual:** Success.
- **Why Needed:** Maintenance management depends on accurate inventory consumption.

TC 15: Garage-Inventory Resource: Parts

- **Modules:** Garage, Inventory
- **Scenario:** Record a repair and use Parts.

- **Expected:** `Inventory.parts_count` is reduced by the amount logged in the repair.
- **Actual:** Success.
- **Why Needed:** Asset tracking across modules must be exact.

TC 16: Garage-Inventory Resource: Cash

- **Modules:** Garage, Inventory
- **Scenario:** Record a repair and use Cash.
- **Expected:** Inventory cash balance is deducted accurately.
- **Actual:** Success.
- **Why Needed:** Ensures financial data flows correctly between the "Shop" layer and "Vault" layer.

TC 17: Resource Insufficiency Rejection

- **Modules:** Garage, Inventory
- **Scenario:** Attempt a repair that requires more cash than exists.
- **Expected:** Transaction is blocked; no resource depletion occurs.
- **Actual:** Success (Correctly Reverted).
- **Why Needed:** Prevents "Negative Balance" or corrupted inventory states.

TC 18: Repair History Logging

- **Modules:** Garage
- **Scenario:** Perform a repair and view history.
- **Expected:** History contains car name, parts, tools, and cash used.
- **Actual:** Success.
- **Why Needed:** Auditability requirement for all maintenance actions.

TC 19: Comprehensive Inventory Status Report

- **Modules:** Inventory

- **Scenario:** Update cash, parts, and cars, then request full status.
- **Expected:** Report correctly aggregates all three data types into a single output.
- **Actual:** Success.
- **Why Needed:** Ensures the "Report" module correctly aggregates data from disparate internal storage variables.

TC 20: High Skill Overflow/Expert Injection

- **Modules:** CrewManagement, Registration
- **Scenario:** Manually set a skill above 100 (e.g., 999).
- **Expected:** System accepts high expertise values for expert data tool purposes.
- **Actual:** Success.
- **Why Needed:** Confirms that there are no hidden/low caps that restrict data tool flexibility.

TC 21: The "Success Loop" Workflow

- **Modules:** All
- **Scenario:** Register A & B → Race A wins → Check Leaderboard shows A as top.
- **Expected:** A successful chain of multi-module updates.
- **Actual:** Success.
- **Why Needed:** Validates the core integration path of the entire software.

TC 22: Resource-to-Skill Full Pipeline

- **Modules:** All
- **Scenario:** Register Han → Add Cash → Repair Car → Check Stats still correlate.
- **Expected:** Changes to inventory don't corrupt the driver's person data.
- **Actual:** Success.
- **Why Needed:** Testing for "side-effects" between unrelated module actions.

TC 23: Ghost Participant Integration Crash

- **Modules:** RaceManagement, Registration
- **Scenario:** Reporting a win for a driver who was never registered.
- **Expected:** Graceful handling or error message.
- **Actual: FAILED (System Crashed - TypeError).**
- **Why Needed:** Exposes total lack of boundary validation between modules.

TC 24: Negative Skill Level Injection

- **Modules:** CrewManagement, Registration
- **Scenario:** Manually setting a skill level to -100.
- **Expected:** Rejection or lower-bound clamping.
- **Actual: FAILED (Logic Breach).** System accepted the negative value.
- **Why Needed:** Found a weakness in logic validation.

TC 25: Negative Inventory Injection

- **Modules:** Inventory
- **Scenario:** Manually setting cash to -5000.
- **Expected:** Handled as debt or rejected.
- **Actual: FAILED (Financial Vulnerability).** Accepted without warning.
- **Why Needed:** Discovers lack of sanity checks in the asset module.

TC 26: Duplicate Entity ID Collision

- **Modules:** Registration
- **Scenario:** Registering "Dom" twice with different roles.
- **Expected:** Rejection and no data corruption of first entry.
- **Actual:** Success (Rejected).
- **Why Needed:** Essential for data isolation.

TC 27: Mission For Non-Existent Member

- **Modules:** MissionPlanning, Registration
- **Scenario:** Starting a mission for a "Ghost" member.
- **Expected:** Silent failure (False) without crash.
- **Actual:** Success.
- **Why Needed:** Ensures that role retrieval doesn't crash on null.

TC 28: Ghost Car Maintenance Logging

- **Modules:** Garage, Inventory
- **Scenario:** Repairing a car that isn't in the inventory.
- **Expected:** rejection since car is non-existent.
- **Actual: FAILED (Integrity Failure).** System logged the repair anyway.
- **Why Needed:** Exposes data integrity weakness between Garage and Inventory.

TC 29: Null-Name Administrative Failure

- **Modules:** Registration
- **Scenario:** Registering an empty string ("") as a name.
- **Expected:** Rejection.
- **Actual:** Success (Rejected).
- **Why Needed:** Boundary check for empty inputs.

TC 30: Vacuum State Leaderboard

- **Modules:** Leaderboard
- **Scenario:** View leaderboard when 0 members are registered.
- **Expected:** Empty list, no crash.
- **Actual:** Success.
- **Why Needed:** Null state robustness.

TC 31: Memory Reference Status Leak

- **Modules:** Inventory
- **Scenario:** Directly mutating the list returned by `Inventory.get_status()`.
- **Expected:** Original inventory remains unchanged.
- **Actual: FAILED (Architectural Weakness).**
- **Why Needed:** Proves that the "Integrity Boundary" between modules is physically broken in memory.

TC 32: Role Case-Sensitivity Fragility

- **Modules:** MissionPlanning, Registration
- **Scenario:** Register as "driver" and attempt "delivery".
- **Expected:** Standardized matching.
- **Actual: FAILED (Logic Error).** rejection due to strict string matching.
- **Why Needed:** Shows that the system is too specific and fails on simple capitalisation.

Fixes

fix1

the record race function inside race was supposed to retrieve a drivers skill level and increment it, but when a ghost driver is passed as the winner, it does `None + 1` , causing an error

fix2

negative values werent checked for in skill level values

fix3

negative values werent checked for in inventory assets

fix 4

fixed issue where repairs could be logged for cards that didnt exist

fix 5

fixed memory leak in inventory where get was returning its internal self.cars list

fix 6

fixed issue where strings werent normalized before testing for equality in missions

```
Dataview (inline field
'=====
===== test session starts
=====
=====
platform linux -- Python 3.14.3, pytest-9.0.2, pluggy-
1.6.0
rootdir:
/home/anivarth/college/sem4/dass/as2/dass_as2/integrati
on/code/tests
collected 31 items
```

```
test_integration.py .....FFF..F..FF
[100%]
```

```
=====
===== FAILURES
=====
=====
```

```
_____ test_race_unregistered_driver
```

```
setup_system = (<registration.Registration object at
```

```
0x7fc759daf410>, <inventory.Inventory object at
0x7fc75931e180>, <crew_managemen...at 0x7fc75931e0f0>,
<missions.MissionPlanning object at 0x7fc75931e060>,
<garage.Garage object at 0x7fc75931dfd0>, ...)
```

```
def test_race_unregistered_driver(setup_system):
    reg, _, crew, racing, _, _, _ = setup_system
    reg.register("Dom", "Driver", 10)
    # If the system crashes here, Pytest will show
it as a FAIL
> racing.record_race("Dom", "C1", "Ghost", "C2",
"Ghost")
```

```
test_integration.py:197:
```

```
-----
-----
-----
```

```
self = <race.RaceManagement object at 0x7fc75931e0f0>,
d1 = 'Dom', c1 = 'C1', d2 = 'Ghost', c2 = 'C2',
winner_name = 'Ghost'
```

```
def record_race(self, d1, c1, d2, c2, winner_name):
    curr_skill =
self.crew.registration.get_skill(winner_name)
> self.crew.registration.set_skill(winner_name,
curr_skill + 1)
```

```
^^^^^^^^^^^^^^^^
```

```
E      TypeError: unsupported operand type(s) for +:
'NoneType' and 'int'
```

```
../race.py:7: TypeError
```

```
_____ test_negative_skill_injection
```

```
setup_system = (<registration.Registration object at
0x7fc75941eb50>, <inventory.Inventory object at
0x7fc75941ff50>, <crew_managemen...at 0x7fc75941d650>,
<missions.MissionPlanning object at 0x7fc75941efd0>,
<garage.Garage object at 0x7fc75941e4d0>, ...)
```

```
def test_negative_skill_injection(setup_system):
    reg, _, crew, _, _, _, _ = setup_system
    reg.register("Dom", "Driver", 10)
    crew.edit_skill("Dom", -100)
    # System should NOT allow negative skill level
> assert reg.get_skill("Dom") >= 0, "Security
Failure: Negative skill allowed!"
E       AssertionError: Security Failure: Negative
skill allowed!
E       assert -100 >= 0
E       + where -100 = get_skill('Dom')
E       + where get_skill =
<registration.Registration object at
0x7fc75941eb50>.get_skill
```

test_integration.py:205: AssertionError

_____ test_negative_inventory_injection

```
setup_system = (<registration.Registration object at
0x7fc75941e1d0>, <inventory.Inventory object at
0x7fc75941e750>, <crew_managemen...at 0x7fc75941e7d0>,
<missions.MissionPlanning object at 0x7fc75941ef50>,
<garage.Garage object at 0x7fc75941d750>, ...)
```

```
def
test_negative_inventory_injection(setup_system):
    _, inv, _, _, _, _, _ = setup_system
```

```
        inv.edit_cash(-5000)
        # System should NOT allow negative cash balance
>        assert inv.get_status()["cash"] >= 0,
"Financial Failure: Negative cash allowed!"
E        AssertionError: Financial Failure: Negative
cash allowed!
E        assert -5000 >= 0
```

test_integration.py:212: AssertionError

_____ test_repair_non_existent_car

setup_system = (<registration.Registration object at
0x7fc7595b4710>, <inventory.Inventory object at
0x7fc7593339a0>, <crew_managemen...at 0x7fc7595b43b0>,
<missions.MissionPlanning object at 0x7fc7595b42f0>,
<garage.Garage object at 0x7fc7595b4230>, ...)

```
def test_repair_non_existent_car(setup_system):
    _, inv, _, _, _, gar, _ = setup_system
    inv.edit_cash(1000)
    inv.edit_tools(10)
    # System should REFUSE repairs for non-existent
cars
    result = gar.record_repair("ImaginaryCar", 1,
0, 100)
>    assert result is False, "Integrity Failure:
Repair logged for non-existent vehicle!"
E        AssertionError: Integrity Failure: Repair
logged for non-existent vehicle!
E        assert True is False
```

test_integration.py:235: AssertionError

_____ test_car_data_mutation_isolation

```
setup_system = (<registration.Registration object at
0x7fc7595c3bb0>, <inventory.Inventory object at
0x7fc759332200>, <crew_managemen...at 0x7fc7595c3c50>,
<missions.MissionPlanning object at 0x7fc75953fbf0>,
<garage.Garage object at 0x7fc75953f9b0>, ...)
```

```
def test_car_data_mutation_isolation(setup_system):
    _, inv, _, _, _, _, _ = setup_system
    inv.add_car("Charger")
    status = inv.get_status()
    # If the system is secure, mutating the report
    should NOT leak into the fleet
    status["cars"].append("LeakedCar")
> assert "LeakedCar" not in inv.cars, "Memory
Leak: Internal state modified via status report!"
E       AssertionError: Memory Leak: Internal state
modified via status report!
E       assert 'LeakedCar' not in ['Charger',
'LeakedCar']
E       + where ['Charger', 'LeakedCar'] =
<inventory.Inventory object at 0x7fc759332200>.cars
```

```
test_integration.py:254: AssertionError
```

```
test_role_case_sensitivity
```

```
setup_system = (<registration.Registration object at
0x7fc7595c3cf0>, <inventory.Inventory object at
0x7fc759331ef0>, <crew_managemen...at 0x7fc7595c3d90>,
<missions.MissionPlanning object at 0x7fc75953eb70>,
<garage.Garage object at 0x7fc75953eed0>, ...)
```

```

def test_role_case_sensitivity(setup_system):
    reg, _, _, _, miss, _, _ = setup_system
    reg.register("Lower", "driver", 10)
    # If the system is robust, it should handle
casing (Driver == driver)
>         assert miss.start_mission("Lower", "delivery")
is True, "Logic Failure: Mission rejected due to case
sensitivity!"
E         AssertionError: Logic Failure: Mission rejected
due to case sensitivity!
E         assert False is True
E         +   where False = start_mission('Lower',
'delivery')
E         +   where start_mission =
<missions.MissionPlanning object at
0x7fc75953eb70>.start_mission

```

```

test_integration.py:261: AssertionError
=====
===== short test summary info
=====
=====
FAILED
test_integration.py::test_race_unregistered_driver -
TypeError: unsupported operand type(s) for +:
'NoneType' and 'int'
FAILED
test_integration.py::test_negative_skill_injection -
AssertionError: Security Failure: Negative skill
allowed!
FAILED
test_integration.py::test_negative_inventory_injection
- AssertionError: Financial Failure: Negative cash
allowed!
FAILED
test_integration.py::test_repair_non_existent_car -
AssertionError: Integrity Failure: Repair logged for

```


non-existent vehicle!

FAILED

test_integration.py::test_car_data_mutation_isolation -
AssertionError: Memory Leak: Internal state modified
via status report!

FAILED test_integration.py::test_role_case_sensitivity
- AssertionError: Logic Failure: Mission rejected due
to case sensitivity!

=====

===== 6 failed, 25 passed in 0.14s

=====

====='): Error:

-- PARSING FAILED -----

> 1 |

=====

===== test session starts

=====

=====

| ^

2 | platform linux -- Python 3.14.3, pytest-9.0.2,
pluggy-1.6.0

3 | rootdir:

/home/anivarth/college/sem4/dass/as2/dass_as2/integrati
on/code/tests

Expected one of the following:

('', 'null', boolean, date, duration, file link, list
('[1, 2, 3]'), negated field, number, object ('{ a: 1,
b: 2 }'), string, variable